LMAD Schedules

Cosmin E. Oancea
cosmin.oancea@diku.dk

Department of Computer Science (DIKU)
University of Copenhagen

1st of May 2025, Presentation for Mary's Group

Motivation

- from LEGO: “Instead of reordering computation to improve memory access patterns, an alternative approach is to change the layout of data in memory to more closely match the order in which the computation accesses it”.
- from “A toxonomy of scheduling languages”, future directions:
  - apply the lessons learned in DSL codegen to full languages
  - provide guarantees for the correctness of the schedule
  - raise the level of abstraction by making schedules accessible to domain experts under a “gentle” learning curve; states the idea of a language of “slices” from which a schedule is infered.
  - ideally, allow the user to express the cost property of a schedule and provide verification that the intended cost is respected, e.g., redundant computation or sliding window.
  - move scheduling signals across the compilation pipeline
  - automatically create schedules for later compilation stages

This presentation argues that it is feasible to convincingly demonstrate all these in a future paper that requires a reasonable amount of work.

Motivation

Halide: Bird's Eye View

Schedules and LMADs

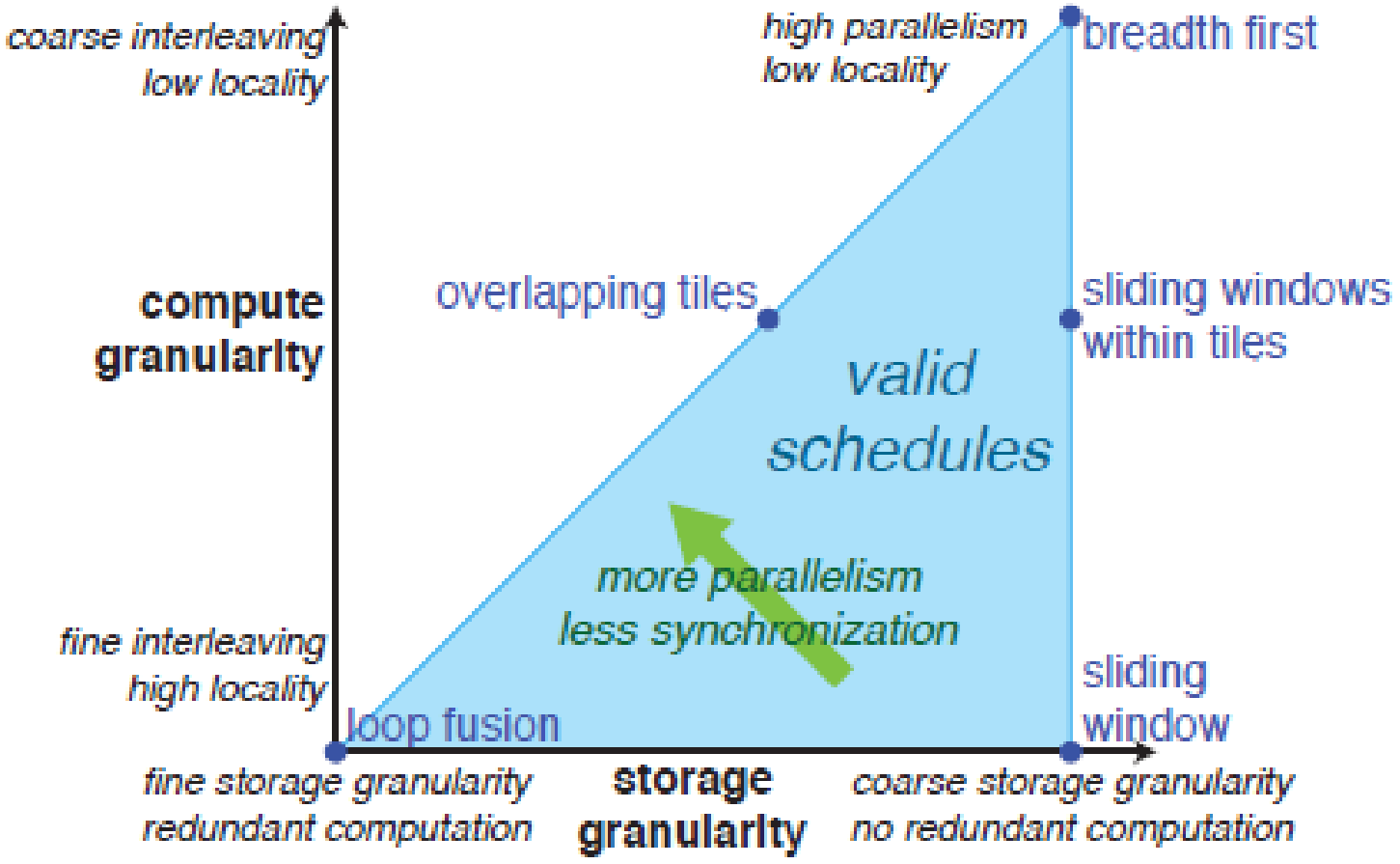
Case Study: Matrix Multiplication

Case Study: NW (?) requires Jagged Arrays & Poly Verification

Case Study: SpMV Smart Schedule (?) requires Jagged Arrays

Automatic Generation of Schedules: AD of Matrix Multiplication

Halide: Tradeoff Space



The tradeoff space is organized along three axes:

- redundant computation
- degree of parallelism mapped to hardware
- locality of reference

Comparison between Optimization Recipes

Strategy	Parallelism	Max Reuse Dist.	Work Ampl.
Breadth-First	3072×2046	$3072 \times 2048 \times 3$	$1.0\times$
Full Fusion	3072×2046	3×3	$2.0\times$
Sliding Window	3072	$3072 \times (3+3)$	$1.0\times$
Tiled	3072×2046	$34 \times 32 \times 3$	$1.0625\times$
Sliding Tiles	$3072 \times 2048/8$	$3072 \times (3+3)$	$1.25\times$

Comparison between Optimization Recipes

Strategy	Parallelism	Max Reuse Dist.	Work Ampl.
Breadth-First	3072×2046	$3072 \times 2048 \times 3$	$1.0\times$
Full Fusion	3072×2046	3×3	$2.0\times$
Sliding Window	3072	$3072 \times (3+3)$	$1.0\times$
Tiled	3072×2046	$34 \times 32 \times 3$	$1.0625\times$
Sliding Tiles	$3072 \times 2048/8$	$3072 \times (3+3)$	$1.25\times$

Which of the five schedules is the one below?

```
1  blurx: split x by 4 → x_o , x_i
2         vectorize: x_i
3         store at out.x_o
4         compute at out.y_i
5  out:  split x by 4 → x_o , x_i
6        split y by 4 → y_o , y_i
7        reorder: y_o , x_o , y_i , x_i
8        parallelize: y_o
9        vectorize:   x_i
```

The algorithmic Specification is:

```
inp:[M+2][N+2]int;   blur: [M+2][N]int;   out: [M][N]int
Func blurf(y,x) = inp[y,x]  + inp[y,x+1]  + inp[y,x+2]
Func outf (y,x) = blurf(y,x)+ blurf(y+1,x)+ blurf(y+2,x)
```

Halide: Compiler Pipeline

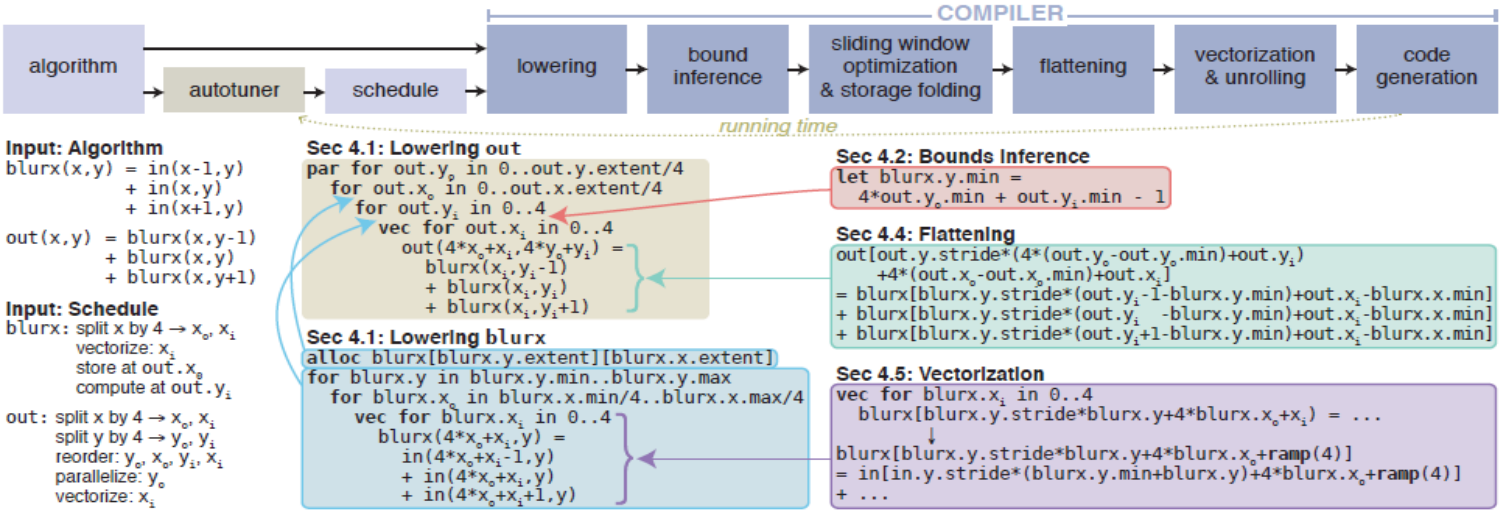


Figure 5. Our compiler is driven by an autotuner, which stochastically searches the space of valid schedules to find a high performance implementation of the given Halide program. The core of the compiler lowers a functional representation of an imaging pipeline to imperative code using a *schedule*. It does this by first constructing a loop nest producing the final stage of the pipeline (in this case `out`), and then recursively injecting the storage and computation of earlier stages of the pipeline at the loop levels specified by the schedule. The locations and sizes of regions computed are symbolic at this point. They are resolved by the subsequent bound inference pass, which injects interval arithmetic computations in a preamble at each loop level that set the region produced of each stage to be at least as large as the region consumed by subsequent stages. Next, sliding window optimization and storage folding remove redundant computation and excess storage where the storage granularity is above the compute granularity. A simple flattening transform converts multidimensional coordinates in the infinite domain of each function into simple one-dimensional indices relative to the base of the corresponding buffer. Vectorization and unrolling passes replace loops of constant with k scheduled as *vectorized* or *unrolled* with the corresponding k-wide vector code or k copies of the loop body. Finally, backend code generation emits machine code for the scheduled pipeline via LLVM.

Halide: Compiler Pipeline

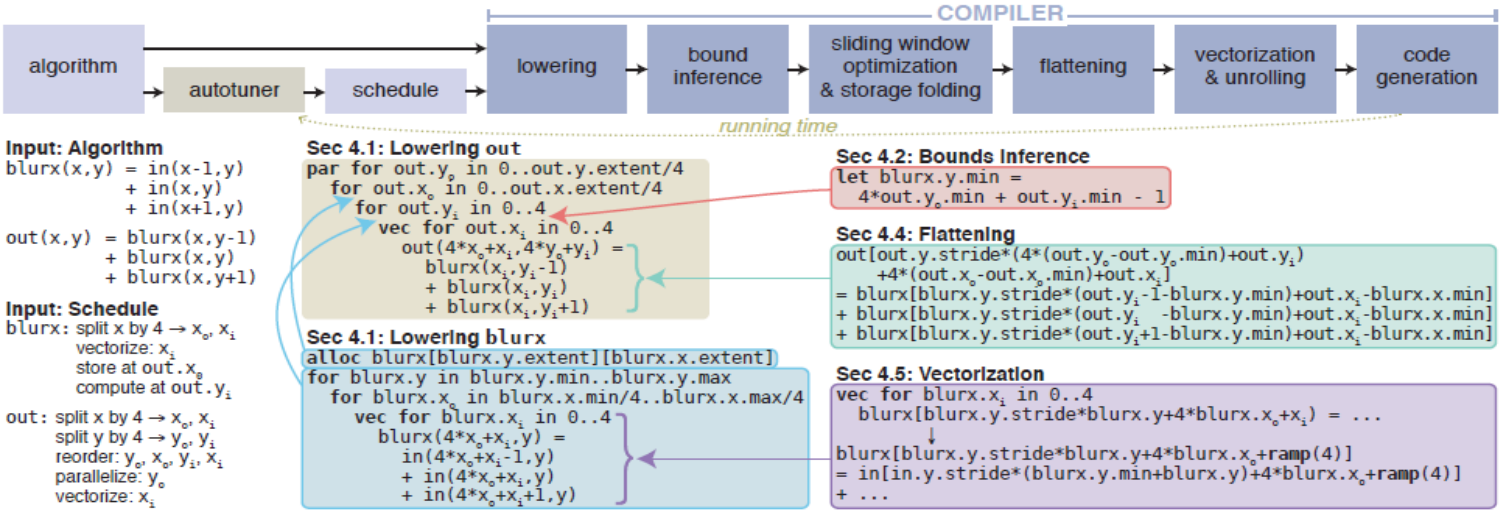


Figure 5. Our compiler is driven by an autotuner, which stochastically searches the space of valid schedules to find a high performance implementation of the given Halide program. The core of the compiler lowers a functional representation of an imaging pipeline to imperative code using a *schedule*. It does this by first constructing a loop nest producing the final stage of the pipeline (in this case `out`), and then recursively injecting the storage and computation of earlier stages of the pipeline at the loop levels specified by the schedule. The locations and sizes of regions computed are symbolic at this point. They are resolved by the subsequent bound inference pass, which injects interval arithmetic computations in a preamble at each loop level that set the region produced of each stage to be at least as large as the region consumed by subsequent stages. Next, sliding window optimization and storage folding remove redundant computation and excess storage where the storage granularity is above the compute granularity. A simple flattening transform converts multidimensional coordinates in the infinite domain of each function into simple one-dimensional indices relative to the base of the corresponding buffer. Vectorization and unrolling passes replace loops of constant with k scheduled as *vectorized* or *unrolled* with the corresponding k-wide vector code or k copies of the loop body. Finally, backend code generation emits machine code for the scheduled pipeline via LLVM.

- the "where to store and where to compute" rationale is not very intuitive
- the schedule as a whole is not intuitive even for compiler experts
- the code generation technique is a bit muddy in places, e.g.,
  the costs/trade-offs are not verified and are not visible in the optimization recipe
  is it guaranteed that the resulted schedule is equivalent with the specification?

Motivation

Halide: Bird's Eye View

Schedules and LMADs

Case Study: Matrix Multiplication

Case Study: NW (?) requires Jagged Arrays & Poly Verification

Case Study: SpMV Smart Schedule (?) requires Jagged Arrays

Automatic Generation of Schedules: AD of Matrix Multiplication

LMADs: an Intuitive Representation of Complex Slicing?

$$\tau + [\overline{(n:s)}^{q}] \equiv \{\tau + i_1 \cdot s_1 + \ldots + i_q \cdot s_q \mid 0 \leq i_1 < n_1, \ldots, 0 \leq i_q < n_q\}$$

- a structured representation of a linearized set of mem refs
- the strides $\bar{s}$ are total
- Many uses:
  - more general slicing than APL's triplets (popular in Python)
  - Index Fun: maps delayed arrays to memory (no manifestation)
  - AutoPar: LMAD summaries "form" dimensions according to how the (flat) memory is being used;
  - **A useful piece in the specification of High-Level Schedules?**
  - we will extend the LMAD rep. with a set of signals attached to each dimension, e.g., slide, shared memory, sequential, etc.

$$map \;:\; [n]\alpha \to (\alpha \to \beta) \to [n]\beta$$
$$map\ [a_0, \ldots, a_{n-1}]\ f \;=\; [f\ a_0, \ldots, f\ a_{n-1}]$$

$$map2 \;:\; [n]\alpha_1 \to [n]\alpha_2 \to (\alpha_1 \to \alpha_2 \to \beta) \to [n]\beta$$
$$map2\ [a_0, \ldots, a_{n-1}]\ [b_0, \ldots, b_{n-1}]\ f \;=\; [f\ a_0\ b_0, \ldots, f\ a_{n-1}\ b_{n-1}]$$

Halide: Fully-Fused Schedule

High-Level specification: arrays represented as index functions:

```
inp:[M+2][N+2]int;   blur: [M+2][N]int;   out: [M][N]int
Func blurf(y,x) = inp[y,x]  + inp[y,x+1]  + inp[y,x+2]
Func outf (y,x) = blurf(y,x)+ blurf(y+1,x)+ blurf(y+2,x)
```

Fully-Fused Schedule

```
alloc out[M][N]
for each y < M:
  for each x < N:
    alloc blur[3]
    for each k < 3:
      blur[k] = in[y+k][x] + in[y+k][x+1] + in[y+k][x+2]
    out[y][x] = blur[0] + blur[1] + blur[2]
```

\+ exploits all parallelism

\+ minimizes reuse distance on `blur`

\- high redundant-computation overhead, i.e., 3×

LMAD: Fully-Fused Schedule

```
blur_o = map (0..M+2) (\y -> map (0..N) (\x->
           inp[y,x] + inp[y,x+1] + inp[y,x+2] ) )
blur = fuseSched(blur_o, 0 + [(M:N),(N:1),(3:N:"seq,reg")])
out  = map (0.. M) (\ y -> map (0..N) (\x ->
           blur[y,x] + blur[y+1,x] + blur[y+2,x] ) )
```

LMAD: Fully-Fused Schedule

```
blur_o = map (0..M+2) (\y -> map (0..N) (\x->
           inp[y,x] + inp[y,x+1] + inp[y,x+2] ) )
blur = fuseSched(blur_o, 0 + [(M:N),(N:1),(3:N:"seq,reg")])
out  = map (0.. M) (\ y -> map (0..N) (\x ->
           blur[y,x] + blur[y+1,x] + blur[y+2,x] ) )
```

LMAD Fully-Fused Schedule: $L^{blur} = 0 + [(M : N), (N : 1), (3 : N : "seq, reg")]$

- logical [M][N][3] & original [M+2][N] $\Rightarrow 3\times$ overhead
- the 1st and 3rd dimensions are overlapping, their extent: M+2
- Logical index $[y, x, k]$ maps to original index $[y + k, x]$, i.e., $L^{bur}(y, x, k) = (y + k) \cdot N + x$

LMAD: Fully-Fused Schedule

```
blur_o = map (0..M+2) (\y -> map (0..N) (\x->
           inp[y,x] + inp[y,x+1] + inp[y,x+2] ) )
blur = fuseSched(blur_o, 0 + [(M:N),(N:1),(3:N:"seq,reg")])
out  = map (0.. M) (\ y -> map (0..N) (\x ->
           blur[y,x] + blur[y+1,x] + blur[y+2,x] ) )
```

LMAD Fully-Fused Schedule: $L^{blur} = 0 + [(M : N), (N : 1), (3 : N : "seq, reg")]$

- logical `[M][N][3]` & original `[M+2][N]` $\Rightarrow 3\times$ overhead
- the 1st and 3rd dimensions are overlapping, their extent: M+2
- Logical index $[y, x, k]$ maps to original index $[y + k, x]$, i.e., $L^{bur}(y, x, k) = (y + k) \cdot N + x$

Step 1: Transform the loop structure of `blur` according to LMAD:

```
blur' = map (0..M) (\i -> map (0..N) (\j-> map (0..3) (\k ->
           inp[y+k,x] + inp[y+k,x+1] + inp[y+k,x+2] ) ) )
out  = map (0.. M) (\ y -> map (0..N) (\x ->
           blur'[y,x,0] + blur'[y,x,1] + blur'[y,x,2] ) )
```

LMAD: Fully-Fused Schedule

```
blur_o = map (0..M+2) (\y -> map (0..N) (\x->
           inp[y,x] + inp[y,x+1] + inp[y,x+2] ) )
blur = fuseSched(blur_o, 0 + [(M:N),(N:1),(3:N:"seq,reg")])
out  = map (0.. M) (\ y -> map (0..N) (\x ->
           blur[y,x] + blur[y+1,x] + blur[y+2,x] ) )
```

LMAD Fully-Fused Schedule: $L^{blur} = 0 + [(M : N), (N : 1), (3 : N : "seq, reg")]$

- logical [M][N][3] & original [M+2][N] $\Rightarrow 3\times$ overhead
- the 1st and 3rd dimensions are overlapping, their extent: M+2
- Logical index $[y, x, k]$ maps to original index $[y + k, x]$, i.e., $L^{bur}(y, x, k) = (y + k) \cdot N + x$

Step 1: Transform the loop structure of `blur` according to LMAD:

```
blur' = map (0..M) (\i -> map (0..N) (\j-> map (0..3) (\k ->
           inp[y+k,x] + inp[y+k,x+1] + inp[y+k,x+2] ) ) )
out  = map (0.. M) (\ y -> map (0..N) (\x ->
           blur'[y,x,0] + blur'[y,x,1] + blur'[y,x,2] ) )
```

Step 2: Map/Loop Fusion:

```
out  = map (0.. M) (\ y -> map (0..N) (\x ->
           let blur' = #[sequential] #[regmem] map (0..3)
                 (\k -> inp[y+k,x] + inp[y+k,x+1] + inp[y+k,x+2])
           in  blur'[0] + blur'[1] + blur'[2]
       ) )
```

Halide: Sliding-Window Schedule

High-Level specification: arrays represented as index functions:

```
inp:[M+2][N+2]int;   blur: [M+2][N]int;   out: [M][N]int
Func blurf(y,x) = inp[y,x]  + inp[y,x+1]  + inp[y,x+2]
Func outf (y,x) = blurf(y,x)+ blurf(y+1,x)+ blurf(y+2,x)
```

Sliding-Window Schedule:

```
alloc out[M][N]
alloc blur[3][N]
for y < 2:
  for each x < N:
    blur[y][x] = inp[y][x] + inp[y][x+1] + inp[y][x+2]
for y < M:
  for each x < N:
    blur[(y+2)%3][x] = inp[y+2][x] + inp[y+2][x+1] + inp[y+2][x+2]
    out[y][x] = blur[y%3][x] + blur[(y+1)%3][x] + blur[(y+2)%3][x]
```

Interleaves the computation in a sliding window with out trailing `blurx` by one scanline

- \- loop `for y` is sequential: a row of `out` depends on the last three rows of `blurx`
- \+ okish locality: max distance between the produced & consumed `blur` is 3 scanlines
- \+ no redundant work (each point in `blurx` computed once)

LMAD: Sliding-Window Schedule

```
blur_o = map (0..M+2) (\y -> map (0..N) (\x->
           inp[y,x] + inp[y,x+1] + inp[y,x+2] ) )
blur = fuseSched(blur_o, 0 + [(M:N:"seq"),(3:N:"slide"),(N:1)])
out  = map (0.. M) (\ y -> map (0..N) (\x ->
           blur[y,x] + blur[y+1,x] + blur[y+2,x] ) )
```

LMAD Sliding-Window Schedule: $L^{blur} = [(M : N : "seq"), (3 : N : "slide"), (N : 1)]$

- logical array `[M][3][N]` where first and second dims overlap; their extent is $M + 2$
- first dimension is marked sequential and the second as slide, hence the semantics is: generate a sliding window of 3 rows without introducing redundant execution.
- Logical index $[y, k, x] \Rightarrow$ orig. index $[y + k, x]$, i.e., $L^{bur}(y, k, x) = (y + k) \cdot N + x$

LMAD: Sliding-Window Schedule

```
blur_o = map (0..M+2) (\y -> map (0..N) (\x->
           inp[y,x] + inp[y,x+1] + inp[y,x+2] ) )
blur = fuseSched(blur_o, 0 + [(M:N:"seq"),(3:N:"slide"),(N:1)])
out  = map (0.. M) (\ y -> map (0..N) (\x ->
           blur[y,x] + blur[y+1,x] + blur[y+2,x] ) )
```

LMAD Sliding-Window Schedule: $L^{blur} = [(M : N : "seq"), (3 : N : "slide"), (N : 1)]$

- logical array `[M][3][N]` where first and second dims overlap; their extent is $M + 2$
- first dimension is marked sequential and the second as slide, hence the semantics is: generate a sliding window of 3 rows without introducing redundant execution.
- Logical index $[y, k, x] \Rightarrow$ orig. index $[y + k, x]$, i.e., $L^{bur}(y, k, x) = (y + k) \cdot N + x$

Step 1: Transform the loop structure of `blur` according to LMAD:

```
blur' = map (0..M) (\y -> map (0..3) (\k -> map (0..N) (\x->
           inp[y+k,x] + inp[y+k,x+1] + inp[y+k,x+2] ) ) )
out  = map (0.. M) (\ y -> map (0..N) (\x ->
           blur'[y,0,x] + blur'[y,1,x] + blur'[y,2,x] ) )
```

LMAD: Sliding-Window Schedule

```
blur_o = map (0..M+2) (\y -> map (0..N) (\x->
           inp[y,x] + inp[y,x+1] + inp[y,x+2] ) )
blur = fuseSched(blur_o, 0 + [(M:N:"seq"),(3:N:"slide"),(N:1)])
out  = map (0.. M) (\ y -> map (0..N) (\x ->
           blur[y,x] + blur[y+1,x] + blur[y+2,x] ) )
```

LMAD Sliding-Window Schedule: $L^{blur} = [(M : N : "seq"), (3 : N : "slide"), (N : 1)]$

- logical array [M][3][N] where first and second dims overlap; their extent is $M + 2$
- first dimension is marked sequential and the second as slide, hence the semantics is: generate a sliding window of 3 rows without introducing redundant execution.
- Logical index $[y, k, x] \Rightarrow$ orig. index $[y + k, x]$, i.e., $L^{bur}(y, k, x) = (y + k) \cdot N + x$

Step 1: Transform the loop structure of `blur` according to LMAD:

```
blur' = map (0..M) (\y -> map (0..3) (\k -> map (0..N) (\x->
           inp[y+k,x] + inp[y+k,x+1] + inp[y+k,x+2] ) ) )
out  = map (0.. M) (\ y -> map (0..N) (\x ->
           blur'[y,0,x] + blur'[y,1,x] + blur'[y,2,x] ) )
```

Step 2: Map/Loop Fusion:

```
blur'=loop (blur') = (alloc ([3][N]int)) for k < 2 do
        blur'[k] = map(0..N)(\x-> inp[y+k,x]+inp[y+k,x+1]+inp[y+k,x+2]
(out, _) = loop (out,blur')=(alloc ([M][N]int),blur') for y < M do
    let (blur_row, out_row) = map (0..N) (\ x ->
          let bl_elm = inp[y+2,x] + inp[y+2,x+1] + inp[y+2,x+2]
          in  ( bl_elm, blur'[y%3][x] + blur'[(y+1)%3][x] + bl_elm ))
    let blur'[(y+2)%3] = blur_row let out[y] = out_row in (blur',out)
```

Halide: Sliding-Window-Within-Tiles Schedule

High-Level specification: arrays represented as index functions:

```
inp:[M+2][N+2]int;   blur: [M+2][N]int;   out: [M][N]int
Func blurf(y,x) = inp[y,x]  + inp[y,x+1]  + inp[y,x+2]
Func outf (y,x) = blurf(y,x)+ blurf(y+1,x)+ blurf(y+2,x)
```

Sliding-Window-Within-Tiles Schedule:

```
alloc out[M][N]
for each yy in 0 .. M-1 by 8:
  alloc blur[3][N]
  for y < 2:
   for each x < N
    blur[y][x] = inp[yy+y][x] + inp[yy+y][x+1] + inp[yy+y][x+2]
  for y < 8:
   for each x < N:
    blur[(y+2)%3][x]=inp[yy+y+2][x]+inp[yy+y+2][x+1]+inp[yy+y+2][x+2]
    out[yy+y][x] = blur[y%3][x] + blur[(y+1)%3][x]+ blur[(y+2)%3][x]
```

"On modern X86 it is −10% to 10% faster than overlapped tiling"

+ sequentializes only `for y in 2+8`
+ good locality (prod-consumer reuse distance: 3 scanlines)
- sacrifices 2 scanlines of redundant work for every 8 scanlines.

LMAD: Sliding-Window-Within-Tiles Schedule

LMAD Tiled-Sliding-Window Schedule assuming $M = m \cdot T_m$:

$L^{blur} = [(m : T_m \cdot N), (T_m : N : "seq"), (3 : N : "slide, shmem"), (N : 1)]$

$L^{out} = [(m : T_m \cdot N), (T_m : N : "seq"), (N : 1)]$

- logical array $[m][T_m][3][N]$ where 2nd and 3rd dims overlap; their extent is $T_m + 2$
- Logical index $[ty, yy, k, x] \Rightarrow$ original index $[ty \cdot T_m + yy + k, x]$, i.e., $L^{bur}(ty, yy, k, x) = ty \cdot T_m \cdot N + (yy + k) \cdot N + x$

LMAD: Sliding-Window-Within-Tiles Schedule

LMAD Tiled-Sliding-Window Schedule assuming $M = m \cdot T_m$:

$L^{blur} = [(m : T_m \cdot N), (T_m : N : "seq"), (3 : N : "slide, shmem"), (N : 1)]$

$L^{out} = [(m : T_m \cdot N), (T_m : N : "seq"), (N : 1)]$

- logical array $[m][T_m][3][N]$ where 2nd and 3rd dims overlap; their extent is $T_m + 2$
- Logical index $[ty, yy, k, x] \Rightarrow$ original index $[ty \cdot T_m + yy + k, x]$, i.e., $L^{bur}(ty, yy, k, x) = ty \cdot T_m \cdot N + (yy + k) \cdot N + x$

```
blur' = map (0..m) (\ty -> map (0..T_m) (\yy -> map (0..3) (\k ->
          let y = ty*T_m + yy + k
          in map (0..N) (\x-> inp[y,x] + inp[y,x+1] + inp[y,x+2]))))
out   = map (0..m) (\ty -> map (0..T_m) (\yy -> map (0..N) (\x ->
          blur'[ty,yy,0,x] + blur'[ty,yy,1,x] + blur'[ty,yy,2,x])))
```

LMAD: Sliding-Window-Within-Tiles Schedule

LMAD Tiled-Sliding-Window Schedule assuming $M = m \cdot T_m$:

$L^{blur} = [(m : T_m \cdot N), (T_m : N : "seq"), (3 : N : "slide, shmem"), (N : 1)]$

$L^{out} = [(m : T_m \cdot N), (T_m : N : "seq"), (N : 1)]$

- logical array $[m][T_m][3][N]$ where 2nd and 3rd dims overlap; their extent is $T_m + 2$
- Logical index $[ty, yy, k, x] \Rightarrow$ original index $[ty \cdot T_m + yy + k, x]$, i.e., $L^{bur}(ty, yy, k, x) = ty \cdot T_m \cdot N + (yy + k) \cdot N + x$

```
blur' = map (0..m) (\ty -> map (0..T_m) (\yy -> map (0..3) (\k ->
          let y = ty*T_m + yy + k
          in map (0..N) (\x-> inp[y,x] + inp[y,x+1] + inp[y,x+2]))))
out   = map (0..m) (\ty -> map (0..T_m) (\yy -> map (0..N) (\x ->
          blur'[ty,yy,0,x] + blur'[ty,yy,1,x] + blur'[ty,yy,2,x])))
```

Step 2: Map/Loop Fusion:

```
let out' = map (0..m) (\ty ->
  let blur'=loop (blur') = (alloc ([3][N]int)) for k < 2 do
      let y = ty*T_m + k in
      blur'[k] = map(0..N)(\x-> inp[y,x]+inp[y,x+1]+inp[y,x+2])
  let (out_tile, _) = loop (out,blur')=(alloc ([T_m][N]int),blur')
    for yy < T_m do
      let y = ty*T_m + yy
      let (b_row, o_row) = map (0..N) (\ x ->
          let bl_elm = inp[y+2,x] + inp[y+2,x+1] + inp[y+2,x+2]
          in  ( bl_elm, blur'[y%3][x] + blur'[(y+1)%3][x] + bl_elm ))
      let blur'[(y+2)%3]=b_row let out_tile[y]=o_row in (blur',out_til
let out = flatten out'
```

Halide: Overlapped-Tiling Schedule

High-Level specification: arrays represented as index functions:

```
inp:[M+2][N+2]int;   blur: [M+2][N]int;   out: [M][N]int
Func blurf(y,x) = inp[y,x]  + inp[y,x+1]  + inp[y,x+2]
Func outf (y,x) = blurf(y,x)+ blurf(y+1,x)+ blurf(y+2,x)
```

Overlapped-Tiling Schedule assumes $M = m \cdot T_m$ and $N = n \cdot T_n$:

```
alloc out[m·T_m][n·T_n]
for each ty < m:
  for each tx < n:
    alloc blur[T_m + 2][T_n]
    for each yy < T_m + 2:
      for each xx < T_n:
        y = ty*T_m+yy;   x = tx*T_n+xx;
        blur[y][x] = in[y][x] + in[y][x+1] + in[y][x+2]
    for each yy < T_m:
      for each xx < T_n:
        y = ty*T_m+yy;   x = tx*T_n+xx;
        out[y][x] = blur[y][x] + blurx[y+1][x] + blur[y+2][x]
```

"On modern X86 this is 10×??? faster than breadth-first"

+ high parallelism degree: both within and across tiles
+ good locality (producer-consumer reuse distance is a tile)
- small amount of redundant work on tile boundaries (ghost)

LMAD: Overlapped-Tiling Schedule

LMAD Overlapped-Tiling Schedule assuming $M = m \cdot T_m,\ N = n \cdot T_n$**:**

$L^{blur} = 0 + [(m : T_m \cdot n \cdot T_n), (n : T_n), (T_m + 2 : n\dot{T}_n : "shmem"), (T_n : 1 : "shmem")]$

$L^{out} = 0 + [(m : T_m \cdot n \cdot T_n), (n : T_n), (T_m : n\dot{T}_n : "shmem"), (T_n : 1 : "shmem")]$

- logical array $[m][n][T_m][T_n]$ where 1st and 3rd dims overlap;
- Logical index $[ty, tx, yy, xx] \Rightarrow$ original flat index $[ty \cdot T_m + yy][tx \cdot T_n + xx]$, i.e., $L^{bur}(ty, tx, yy, xx) = ty \cdot T_m \cdot n \cdot T_n + tx \cdot T_n + yy \cdot n \cdot T_n + xx$

LMAD: Overlapped-Tiling Schedule

LMAD Overlapped-Tiling Schedule assuming $M = m \cdot T_m,\ N = n \cdot T_n$**:**

$L^{blur} = 0 + [(m : T_m \cdot n \cdot T_n), (n : T_n), (T_m + 2 : n\dot{T}_n : "shmem"), (T_n : 1 : "shmem")]$

$L^{out} = 0 + [(m : T_m \cdot n \cdot T_n), (n : T_n), (T_m : n\dot{T}_n : "shmem"), (T_n : 1 : "shmem")]$

- logical array $[m][n][T_m][T_n]$ where 1st and 3rd dims overlap;
- Logical index $[ty, tx, yy, xx] \Rightarrow$ original flat index $[ty \cdot T_m + yy][tx \cdot T_n + xx]$, i.e., $L^{bur}(ty, tx, yy, xx) = ty \cdot T_m \cdot n \cdot T_n + tx \cdot T_n + yy \cdot n \cdot T_n + xx$

```
blur' = map (0..m) (\ty -> map (0..n) (\tx ->
          map (0..Tm+2) (\yy -> map (0..Tn) (\xx ->
              let (y, x) = (ty*Tm + yy, tx*Tn + xx)
              in  inp[y,x] + inp[y,x+1] + inp[y,x+2]))))
out = map (0..m) (\ty -> map (0..n) (\tx ->
        map (0..Tm) (\yy -> map (0..Tn) (\xx ->
          blur'[ty,tx,yy,xx]+blur'[ty,tx,yy+1,xx]+blur'[ty,tx,yy+2,xx]))
```

LMAD: Overlapped-Tiling Schedule

LMAD Overlapped-Tiling Schedule assuming $M = m \cdot T_m,\ N = n \cdot T_n$**:**

$L^{blur} = 0 + [(m : T_m \cdot n \cdot T_n), (n : T_n), (T_m + 2 : n\dot{T}_n : "shmem"), (T_n : 1 : "shmem")]$

$L^{out} = 0 + [(m : T_m \cdot n \cdot T_n), (n : T_n), (T_m : n\dot{T}_n : "shmem"), (T_n : 1 : "shmem")]$

- logical array $[m][n][T_m][T_n]$ where 1st and 3rd dims overlap;
- Logical index $[ty, tx, yy, xx] \Rightarrow$ original flat index $[ty \cdot T_m + yy][tx \cdot T_n + xx]$, i.e., $L^{bur}(ty, tx, yy, xx) = ty \cdot T_m \cdot n \cdot T_n + tx \cdot T_n + yy \cdot n \cdot T_n + xx$

```
blur' = map (0..m) (\ty -> map (0..n) (\tx ->
          map (0..Tm+2) (\yy -> map (0..Tn) (\xx ->
              let (y, x) = (ty*Tm + yy, tx*Tn + xx)
              in  inp[y,x] + inp[y,x+1] + inp[y,x+2]))))
out = map (0..m) (\ty -> map (0..n) (\tx ->
        map (0..Tm) (\yy -> map (0..Tn) (\xx ->
          blur'[ty,tx,yy,xx]+blur'[ty,tx,yy+1,xx]+blur'[ty,tx,yy+2,xx]))
```

Step 2: Trivial Map Fusion:

```
let out'= map (0..m) (\ty -> map (0..n) (\tx ->
            let blur_tile =
              map (0..Tm+2) (\yy -> map (0..Tn) (\xx ->
                let (y, x) = (ty*Tm + yy, tx*Tn + xx)
                in  inp[y,x] + inp[y,x+1] + inp[y,x+2]))
            in map (0..Tm) (\yy -> map (0..Tn) (\xx ->
                 blur'[yy,xx]+blur'[yy+1,xx]+blur'[yy+2,xx]))
          ))
let out[L^out] = out' -- applies original layout in O(1)
```

Motivation

Halide: Bird's Eye View

Schedules and LMADs

Case Study: Matrix Multiplication

Case Study: NW (?) requires Jagged Arrays & Poly Verification

Case Study: SpMV Smart Schedule (?) requires Jagged Arrays

Automatic Generation of Schedules: AD of Matrix Multiplication

LMAD Schedule for Matrix Multiplication

```
def mmm m T_m R_m n T_n R_n q T_q (A: [m·T_m·R_m][q·T_q]f32)
        (B: [q·T_q][n·T_n·R_n]f32) : [m·T_m·R_m][n·T_n·R_n]f32 =
  map (0..m·T_m·R_m) (\i -> map (0..n·T_n·R_n) (\j ->
      loop elm_res = 0.0 for k < q·T_q do
        elm_res = elm_res + A[i,k] * B[k,j]
```

LMAD Overlapped-Tiling Schedule:

$$L^A = 0 + [(m : T_m \cdot R_m \cdot q \cdot T_q), (q : T_q : \text{"}seq\text{"}), (T_m : R_m \cdot q \cdot T_q : \text{"}shmem\text{"}),$$
$$(T_q : 1 : \text{"}seq, shmem\text{"}), (R_m : q \cdot T_q : \text{"}seq, shmem\text{"})]$$
$$L^B = 0 + [(n : T_n \cdot R_n), (q : T_q \cdot n \cdot T_n \cdot R_n : \text{"}seq\text{"}), (T_n : R_n : \text{"}shmem\text{"}),$$
$$(T_q : n \cdot T_n \cdot R_n : \text{"}shmem\text{"}), (R_n : 1 : \text{"}seq, shmem\text{"})]$$
$$L^C = 0 + [(m : T_m \cdot R_m \cdot n \cdot T_n \cdot R_n), (n : T_n \cdot R_n), (T_m : R_m \cdot n \cdot T_n \cdot R_n), (T_n : R_n),$$
$$(R_m : n \cdot T_n \cdot R_n : \text{"}seq, reg\text{"}), (R_n : 1 : \text{"}seq, reg\text{"})]$$

LMAD Schedule for Matrix Multiplication

```
def mmm m T_m R_m n T_n R_n q T_q (A: [m·T_m·R_m][q·T_q]f32)
        (B: [q·T_q][n·T_n·R_n]f32) : [m·T_m·R_m][n·T_n·R_n]f32 =
  map (0..m·T_m·R_m) (\i -> map (0..n·T_n·R_n) (\j ->
      loop elm_res = 0.0 for k < q·T_q do
        elm_res = elm_res + A[i,k] * B[k,j]
```

LMAD Overlapped-Tiling Schedule:

$$L^A = 0 + [(m : T_m \cdot R_m \cdot q \cdot T_q), (q : T_q : \text{"}seq\text{"}), (T_m : R_m \cdot q \cdot T_q : \text{"}shmem\text{"}),$$
$$(T_q : 1 : \text{"}seq, shmem\text{"}), (R_m : q \cdot T_q : \text{"}seq, shmem\text{"})]$$
$$L^B = 0 + [(n : T_n \cdot R_n), (q : T_q \cdot n \cdot T_n \cdot R_n : \text{"}seq\text{"}), (T_n : R_n : \text{"}shmem\text{"}),$$
$$(T_q : n \cdot T_n \cdot R_n : \text{"}shmem\text{"}), (R_n : 1 : \text{"}seq, shmem\text{"})]$$
$$L^C = 0 + [(m : T_m \cdot R_m \cdot n \cdot T_n \cdot R_n), (n : T_n \cdot R_n), (T_m : R_m \cdot n \cdot T_n \cdot R_n), (T_n : R_n),$$
$$(R_m : n \cdot T_n \cdot R_n : \text{"}seq, reg\text{"}), (R_n : 1 : \text{"}seq, reg\text{"})]$$

Inferring Transformation from each LMAD from the current iteration space:

C: $m \cdot T_m \cdot R_m \Rightarrow m \times T_m \times R_m$, $n \cdot T_n \cdot R_n \Rightarrow n \times T_n \times R_n$, $\sigma_C = [0, 3, 1, 4, 2, 5]$

A: $m \cdot T_m \cdot R_m \Rightarrow m \times T_m \times R_m$, $q \cdot T_q \Rightarrow q \times T_q$, $\sigma_A = [0, 3, 1, 4, 2]$

B: $q \cdot T_q \Rightarrow q \times T_q$, $n \cdot T_n \cdot R_n \Rightarrow n \times T_n \times R_n$. $\sigma_B = [2, 0, 3, 1, 4]$ (redundant)

LMAD Schedule for Matrix Multiplication

```
def mmm m T_m R_m n T_n R_n q T_q (A: [m·T_m·R_m][q·T_q]f32)
        (B: [q·T_q][n·T_n·R_n]f32) : [m·T_m·R_m][n·T_n·R_n]f32 =
  map (0..m·T_m·R_m) (\i -> map (0..n·T_n·R_n) (\j ->
      loop elm_res = 0.0 for k < q·T_q do
        elm_res = elm_res + A[i,k] * B[k,j]
```

LMAD Overlapped-Tiling Schedule:

$$L^A = 0 + [(m : T_m \cdot R_m \cdot q \cdot T_q), (q : T_q : "seq"), (T_m : R_m \cdot q \cdot T_q : "shmem"), (T_q : 1 : "seq, shmem"), (R_m : q \cdot T_q : "seq, shmem")]$$

$$L^B = 0 + [(n : T_n \cdot R_n), (q : T_q \cdot n \cdot T_n \cdot R_n : "seq"), (T_n : R_n : "shmem"), (T_q : n \cdot T_n \cdot R_n : "shmem"), (R_n : 1 : "seq, shmem")]$$

$$L^C = 0 + [(m : T_m \cdot R_m \cdot n \cdot T_n \cdot R_n), (n : T_n \cdot R_n), (T_m : R_m \cdot n \cdot T_n \cdot R_n), (T_n : R_n), (R_m : n \cdot T_n \cdot R_n : "seq, reg"), (R_n : 1 : "seq, reg")]$$

Inferring Transformation from each LMAD from the current iteration space:

C: $m \cdot T_m \cdot R_m \Rightarrow m \times T_m \times R_m$, $n \cdot T_n \cdot R_n \Rightarrow n \times T_n \times R_n$, $\sigma_C = [0, 3, 1, 4, 2, 5]$

A: $m \cdot T_m \cdot R_m \Rightarrow m \times T_m \times R_m$, $q \cdot T_q \Rightarrow q \times T_q$, $\sigma_A = [0, 3, 1, 4, 2]$

B: $q \cdot T_q \Rightarrow q \times T_q$, $n \cdot T_n \cdot R_n \Rightarrow n \times T_n \times R_n$. $\sigma_B = [2, 0, 3, 1, 4]$ (redundant)

A consistent optimization recipe can be built:

- stripmine $m \cdot T_m \cdot R_m \Rightarrow m \times T_m \times R_m$
- stripmine $n \cdot T_n \cdot R_n \Rightarrow n \times T_n \times R_n$
- stripmine $q \cdot T_q \Rightarrow q \times T_q$
- interchange obtained loop nest by $\sigma = [0, 3, 6, 1, 4, 7, 2, 5]$, i.e., original structure $[m, T_m, R_m, n, T_n, R_n, q, T_q] \Rightarrow [m, n, q, T_m, T_n, T_q, R_m, R_n]$

Motivation

Halide: Bird's Eye View

Schedules and LMADs

Case Study: Matrix Multiplication

Case Study: NW (?) requires Jagged Arrays & Poly Verification

Case Study: SpMV Smart Schedule (?) requires Jagged Arrays

Automatic Generation of Schedules: AD of Matrix Multiplication

Motivation

Halide: Bird's Eye View

Schedules and LMADs

Case Study: Matrix Multiplication

Case Study: NW (?) requires Jagged Arrays & Poly Verification

Case Study: SpMV Smart Schedule (?) requires Jagged Arrays

Automatic Generation of Schedules: AD of Matrix Multiplication